

Robust Vulnerability Detection across Compilations: LLVM-IR vs. Assembly with Transformer Model

RONY SHIR, Ben-Gurion University of the Negev, Israel

PRIYANKA SURVE, Ben-Gurion University of the Negev, Israel

YUVAL ELOVICI, Ben-Gurion University of the Negev, Israel

ASAF SHABTAI, Ben-Gurion University of the Negev, Israel

Detecting vulnerabilities in binary files is a challenging task in cybersecurity, particularly when source code is unavailable and the compilation process and its parameters are unknown. Existing deep learning-based detection methods often rely on knowing a binary's specific compilation settings, which may limit their ability to perform well on other types of binaries. In this research, we provide a thorough comparison of assembly representation and LLVM-IR to identify which representation is more robust and suitable when compilation parameters are unknown. The choice of representation significantly influences detection accuracy. Another contribution of this paper is the use of CodeBERT, a transformer-based model, as a classification tool for detecting vulnerabilities in scenarios where the compilation process is unknown. This study applies transformer models to the task of multi-class vulnerability detection in the LLVM-IR domain, with a focus on binary-derived representations. While recent research has explored the use of transformers for vulnerability analysis in source code and raw binary instruction streams, systematic evaluation as a classifier at the LLVM-IR level remains limited. Prior work has commonly relied on RNN-based methods, which are considered state-of-the-art for this task; however, these models struggle to capture long-range dependencies effectively. To address this limitation, we extend transformer-based classification to LLVM-IR produced from binaries and provide a comprehensive evaluation in this setting. Our results highlight the potential of this approach to strengthen system security across diverse binary configurations.

CCS Concepts: • **Security and privacy** → **Vulnerability scanners**; • **Computing methodologies** → *Neural networks*; Artificial intelligence.

Additional Key Words and Phrases: LLVM-IR, assembly, vulnerability detection, CodeBERT, transformer, compilation, architecture, compiler, multi-class classification

ACM Reference Format:

Rony Shir, Priyanka Surve, Yuval Elovici, and Asaf Shabtai. 2025. Robust Vulnerability Detection across Compilations: LLVM-IR vs. Assembly with Transformer Model. *Proc. ACM Softw. Eng.* 2, ISSTA, Article ISSTA028 (July 2025), 22 pages. <https://doi.org/10.1145/3728903>

1 Introduction

A software vulnerability is a flaw or weakness in code that can affect programs' behavior, causing them to act unpredictably and providing an entry point for attackers that allows them to gain unauthorized access and perform maliciously [1, 4]. As cyberattacks grow more sophisticated, the

Authors' Contact Information: [Rony Shir](#), Ben-Gurion University of the Negev, beer sheva, Israel, ronyshi@post.bgu.ac.il; [Priyanka Surve](#), Ben-Gurion University of the Negev, beer sheva, Israel, surve@post.bgu.ac.il; [Yuval Elovici](#), Ben-Gurion University of the Negev, beer sheva, Israel, elovici@bgu.ac.il; [Asaf Shabtai](#), Ben-Gurion University of the Negev, beer sheva, Israel, shabtaia@bgu.ac.il.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2025 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 2994-970X/2025/7-ARTISSTA028

<https://doi.org/10.1145/3728903>

need to detect vulnerabilities has become critical for maintaining software integrity [20]. To facilitate vulnerability detection, researchers often lift binaries into intermediate representations (IRs), transforming low-level machine code into higher-level abstractions [11]. Prior research on binary vulnerability detection has used representation languages like assembly [5, 11, 13, 26, 29] and low-level virtual machine intermediate representation (LLVM-IR) [16, 24, 32] to transform binary code into high-level representations and detect vulnerabilities. Assembly's close relationship to machine code provides detailed insights into hardware interactions, aiding in the detection of low-level issues such as buffer overflows. However, assembly's low-level specificity makes it highly sensitive to variations introduced by different compilation settings, where optimizations such as instruction reordering, register allocation changes, and dead-code elimination or different architectures can cause significant changes to its structure [7]. As a result, assembly-based vulnerability detection can be more prone to changes in compilation settings than higher-level representations. In contrast, LLVM-IR provides an architecture-agnostic intermediate representation that retains higher-level program structure, making it more stable than assembly. This stability makes LLVM-IR well-suited for vulnerability detection in multi-setting scenarios where maintaining consistency across different compilation environments is important [16].

In this research, we evaluate the robustness of two types of representation languages – assembly and LLVM-IR – for vulnerability detection in binaries, specifically, in cases where the source code is unavailable. When source code is available for vulnerability analysis, it provides a logical structure and program flow, facilitating analysis [1]. However, in many cases, only binary files are available, complicating efforts to trace the original code structure, especially when unknown compilation parameters and compiler transformations further obscure vulnerabilities and make reverse engineering difficult [1, 30].

Existing deep learning based methods for detecting vulnerabilities in binary files typically rely on uniform compilation settings, i.e., the dataset used for training and testing is compiled using a single set of known compilation parameters. Recent advancements in machine learning, especially transformer models, have shown promising results in capturing complex code patterns, enabling more sophisticated vulnerability detection in binary files [5]. However, in real-world binaries, no uniform compilation settings exist, underscoring the need for datasets that reflect real-world variability. To the best of our knowledge, no binary dataset currently exists that is suitable for multi-class vulnerability detection. This creates a need for compiling high-level source code into binary form for this purpose. Most existing datasets for binary vulnerability detection are either synthetic or developed under controlled conditions, which limits their ability to accurately reflect the diversity and complexity found in real-world binaries [8]. Static analysis techniques often struggle with the inconsistencies introduced by different compilation processes, which usually results in high false positive and negative rates [26]. To address the challenges introduced by varied compilation configurations, current approaches in this domain frequently employ similarity-based methods [12, 13]. Our contribution diverges from these by focusing specifically on identifying which representation—LLVM-IR or assembly—is most suitable for detecting vulnerabilities when compilation parameters are unknown. By leveraging CodeBERT [31], a transformer-based [27] model for vulnerability classification, we can systematically examine the strengths and limitations of each representation language (assembly and LLVM-IR) in effectively handling the variability of binary files produced by different compilation configurations. To address the limitation of unavailability of diverse binary dataset, we utilize the Juliet Test Suite [23] and compiling it under four different parameter configurations to introduce compilation variability that mirrors practical scenarios.

The compiled dataset is then transformed into both assembly and LLVM-IR, providing diverse test cases for assessing the transformer-based vulnerability detection method. Our dataset encompasses 23 distinct vulnerability types and a "not vulnerable" class that will be used for multi-class vulnerability classification task. By leveraging these representations and a fine-tuned transformer model, we aim to evaluate the robustness of vulnerability detection across varied compilation settings in multi-class classification task.

In our study, we used the -O0 optimization level to ensure that the structural differences between LLVM-IR and assembly could be analyzed without interference from optimization-induced transformations. The -O0 level preserves more information, allowing a precise comparison of representations. While higher optimization levels (-O2, -O3) introduce transformations like instruction reordering and loop unrolling, our transformer-based approach focuses on semantic understanding, making it suitable for different optimization settings. The model was fine-tuned separately on LLVM-IR and assembly representations. The results indicate that LLVM-IR provides greater robustness across different compilation setups than assembly, making it a more suitable choice for detecting vulnerabilities in binaries when the specifics of the compilation process are unknown.

To summarize, we make the following key contributions to the field:

- (1) We systematically evaluate the robustness of LLVM-IR and assembly representations using various compilation configurations, in order to determine which is more suitable for detecting vulnerabilities when compilation parameters are unknown.
- (2) We fine-tune a transformer-based classification model (CodeBERT) to detect and classify vulnerabilities in binary files, particularly in scenarios where the source code and compilation parameters are unavailable.
- (3) Employs transformers as the primary classification tool for detecting vulnerabilities in LLVM-IR derived from binary files.
- (4) We introduce a new dataset derived from the Juliet Test Suite, compiled under varying compilation configurations.

The structure of this paper is as follows: Section 2 reviews prior research in this field and identifies the gaps in the literature. In Section 3, we describe the methodology implemented to achieve the research goals. Section 4 discusses the dataset's preparation, compilation environment, and data preprocessing, and Section 5 explains the experimental setup used to test and validate the proposed methodology as well as the research challenges and limitations. Section 6 presents the experimental results and evaluates them based on the defined performance metrics. Finally, in Section 7 we conclude with a discussion of our findings and plans for future research.

2 Related Work

Static analysis techniques for binary code vulnerability detection can be broadly categorized based on the representation language used to describe the binary code. Selecting a representation misaligned with task requirements can directly impact the accuracy and resilience of detection. Two primary representations – assembly and LLVM-IR – dominate this field, each with its own advantages and limitations. A detailed summary of prior work in vulnerability detection domain including the representation language, models used for classification, datasets employed, and whether they focus on single or multiple compilation settings is provided in Table 1.

2.1 Assembly as a Representation Language for Vulnerability Detection

Assembly is widely used in binary vulnerability detection due to its low-level view of machine instructions, which is ideal for detailed analysis of binary interactions with hardware. Its closeness

Table 1. Summary of existing research.

Ref.	Representation Language	Model Used	Dataset Used	Single/Multi Compilation
[32], 2019	LLVM-IR	Six different RNNs	CWE-134,401,191,590 from the National Institute of Standards and Technology (NIST) Software Assurance Reference Dataset (SARD)	Single
[26], 2020	Assembly	Six different RNNs	The source program dataset derived from the NIST SARD	Single
[13], 2020	Assembly	Similarity-based	Open sources and real-world firmware images	Different optimization, architecture and compilers
[29], 2021	Assembly	Fully connected network with softmax function	Juliet Test Suite v1.3 for C/C++ (SARD), ICLR19	Single
[8], 2022	Assembly	RoBERTa and MPNN	CWE-121,190 from Juliet Test Suite and CWE-119 NDSS18 dataset (NIST)	Different OSs
[2], 2022	Assembly	LSTM, BLSTM, BGRU	SARD dataset	Single
[24], 2022	LLVM-IR	RNN, GRU, LSTM	Juliet Test Suite v1.3 for C/C++ (SARD)	Single
[11], 2023	Assembly	BiGRU, attention, GCN	NDSS18, Juliet Test Suite, and a real-world CVE dataset introduced in [6]	Different OSs
[28], 2023	Assembly	Fully connected layer with softmax (after BiGRU, ELMo)	Juliet Test Suite v1.3 for C/C++ and NDSS18	Different OSs
[5], 2023	Assembly	CodeBERT	Juliet Test Suite v1.3 for C/C++	Single
[12], 2023	Assembly	Similarity-based	Open-source software	Different optimization and architecture
[16], 2024	LLVM-IR	LSTM	Juliet Test Suite v1.3 for C/C++	Different OSs
Ours, 2025	LLVM-IR, Assembly	Transformer - CodeBERT	Juliet Test Suite v1.3 for C/C++	Different architectures and compilers

to machine code facilitates the detection of low-level vulnerabilities like buffer overflows and memory corruption.

Several studies have utilized the assembly language for vulnerability detection. In the work that has been done by [2], they extracted syntax information from assembly instructions and used these features to train two deep-learning models for detecting vulnerabilities. BVDetector, introduced by [26] identifies vulnerabilities through program slices of library/API function calls using a bidirectional gated recurrent unit (BGRU) neural network. Hierarchical attention network was developed by [29] for binary software vulnerability detection, based on four components: preprocessor, instruction embedding network, feature extraction network and classifier. These components are built upon BGRU, word-attention module, Text-CNN, and spatial-attention module to highlight key areas.

Multi-level feature fusion model outlined by [28] captures word-level and instruction-level details using ELMo to generate dynamic word vectors with semantic depth. Assembly code segments were normalized by leveraging random embeddings and BGRU was used to extract sequence features. The authors used a weighted fusion approach to evaluate the features' impact on model performance. The work that has been done by [11] utilized sequential and structural learning via recurrent units and graph convolution to mimic program execution. An integrated attention mechanism within their model illustrates the impact of various instructions and their states on the final classification. Additionally, multi-task learning was employed to identify specific vulnerability types. By focusing on capturing both structural and semantic aspects of assembly code, [8] utilizing RoBERTa for basic block embeddings and a message passing neural network (MPNN) for comprehensive code representations. A text-based assembly representation that includes function-spanning contexts and semantics was introduced in [5], where the authors leveraged CodeBERT for high-level vulnerability detection. However, their approach is limited to a single compilation, which may limit generalization

across different instruction sets; in contrast, our proposed method performs vulnerability detection on multiple architectures (ARM and x86-64) and compilers (GCC and Clang) by comparing assembly-based representations with LLVM-IR. This comparison demonstrates the robustness of LLVM-IR by evaluating how well each representation handles compilation variability, leveraging LLVM-IR's architecture-agnostic nature to enhance model adaptability across diverse compilation settings and addressing the limitations of studies that relied solely on assembly text representations.

Assembly language has some drawbacks, given its difficulty in producing insights into software design (e.g., loops) [21]. The same source code, when compiled with different optimizations or for different architectures, can result in widely varying assembly code, although the functionality remains the same. This variability poses challenges in detecting vulnerabilities across multi-compilation settings, as highlighted in [7].

2.2 LLVM-IR as a Representation Language for Vulnerability Detection

LLVM-IR offers higher-level abstraction than assembly and is designed to be architecture-agnostic. It serves as an intermediary between source code and machine code, making it easier to apply machine-independent optimizations [21]. This agnostic nature makes LLVM-IR more robust against differences introduced by varying compilation settings, as it captures the logical structure of the program more clearly than assembly.

Research on LLVM-IR in vulnerability detection from the binary level is relatively limited compared to assembly. The work that has been done in [32] utilized RetDec [3] to lift binary code into LLVM-IR and applied recurrent neural network (RNN) based networks for vulnerability classification. In [24], they extended this work by increasing the dataset size and improving classification models. While these studies demonstrate the potential of LLVM-IR, no research has fully explored the use of transformers as the main classification model for LLVM-IR-based vulnerability detection.

The lack of transformer applications for direct binary vulnerability detection in LLVM-IR and the limited amount of research utilizing this representation highlights critical gaps in the field. Transformers' ability to model long-range dependencies and capture complex patterns makes them highly effective for tasks that require an in-depth understanding of intricate relationships within data like vulnerability detection [22]. In LLVM-IR vulnerability detection from the binary level research, however, transformers have so far been employed primarily for embedding, with long short-term memory (LSTM) networks used for classification [16]. This research aims to address these gaps by employing transformers directly to LLVM-IR for vulnerability detection in binary files, leveraging their strengths to enhance accuracy and robustness in this complex task.

2.3 Multi-Compilation Settings

Another key limitation in existing work is the reliance on single-compilation settings. Many studies, including those using assembly and LLVM-IR as described above, focus on detecting vulnerabilities in binaries compiled with uniform parameters. While these approaches yield high accuracy within controlled environments, they do not account for the variability introduced by real-world compilation processes, which can significantly affect vulnerability detection. To the best of our knowledge, research addressing multi-compilation scenarios mainly focused on similarity-based approaches [12, 13]. Our research was motivated by the gaps identified in prior research on vulnerability detection across different representation languages. Assembly has been the primary representation language due to its low-level insight into hardware interactions, making it effective for detecting vulnerabilities close to machine code, such as buffer overflows and memory corruption. However, prior studies have revealed assembly's significant limitations when handling varied compilation settings, as its sensitivity to compiler transformations can lead to inconsistencies

and reduced accuracy across diverse binaries [7, 21]. On the other hand, although less studied in this domain, LLVM-IR offers potential advantages with its higher-level abstraction and architecture-agnostic nature, which can mitigate the syntactic gaps caused by different compilation environments. Research performed in [24, 32] suggested that LLVM-IR may offer a more stable representation by capturing program structure more consistently, yet the application of advanced classifiers, particularly transformers, remains under-explored for LLVM-IR [24, 32].

This disproportion in exploration between assembly and LLVM-IR prompted our comprehensive evaluation of each representation's robustness, with LLVM-IR hypothesized as a potentially superior choice for achieving stability across diverse binaries. Based on these insights obtained from related work in field of vulnerability detection, we chose to leverage CodeBERT [31] as the classification tool, as it directly models complex patterns and long-range dependencies within code. This approach allows for a comprehensive comparison of representation languages, advancing both understanding and practical applicability in binary vulnerability detection.

Building on prior work in binary vulnerability detection, this research contributes by comparing LLVM-IR and assembly representations across diverse compilation settings, using a transformer as a classifier in the LLVM-IR domain, and releasing a cross-compilation Juliet-based dataset to support future research.

3 Methodology

Our primary objective was to compare the robustness of representation languages for vulnerability detection in binary files when the compilation process is unknown and the source code is unavailable. We sought to determine whether LLVM-IR's architecture-agnostic characteristics improved robustness or assembly's hardware-specific insights enabled more accurate vulnerability detection. This exploratory testing was aimed at comparing how well each representation handled the variations introduced by different architectures and compilers. Additionally, the study investigated a transformer-based model's ability to improve vulnerability detection when used as a classification tool, with LLVM-IR serving as the representation language.

Our methodology consists of three key steps: First, a comprehensive dataset was created by compiling the Juliet Test Suite [23] using four compilation parameter combinations. Next, binary files were converted into two representations – LLVM-IR and assembly – to facilitate comparative analysis. LLVM-IRs were generated using the RetDec tool [3], while assembly code was obtained using objdump and processed with Python scripts [5]. Finally, a transformer-based classification model was fine-tuned separately for each representation, enabling it to learn from both LLVM-IR and assembly structures, allowing us to assess each representation language's effectiveness in vulnerability detection.

Figure 1 provides an overview of the entire methodology, detailing each phase in the process—from data preparation to representation transformation and model training. As can be seen in the figure, we are following a systematic approach to vulnerability detection, designed to accommodate varying compilation scenarios and provide a comprehensive evaluation of each representation's robustness.

Dataset Creation: The dataset creation process began with the compilation of the source code under two architectures (x86-64 and ARM) using two compilers (GCC and Clang) at the O0 optimization level. This resulted in four distinct compiled binary datasets, each representing a unique combination of architecture and compiler configuration - x86-64 with GCC, x86-64 with Clang, ARM with GCC, and ARM with Clang. Once the binaries were compiled, we performed a lifting step to generate two types of representations— assembly and LLVM-IR—for further analysis:

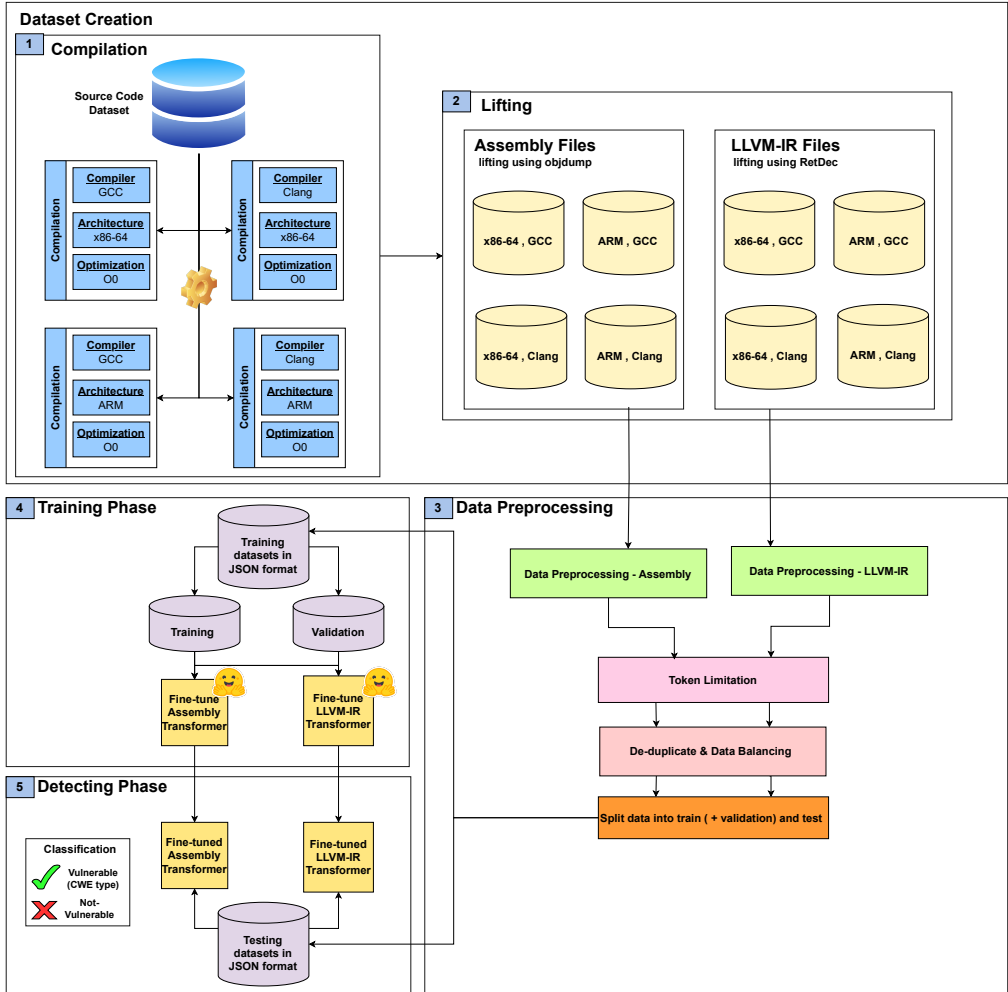


Fig. 1. Methodology.

- **Assembly Dataset:** Each compiled binary is converted to an assembly file using objdump. This generates an assembly dataset comprising four subsets, each corresponding to one of the original compilation configurations (x86-64 with GCC, x86-64 with Clang, ARM with GCC, ARM with Clang).
- **LLVM-IR Dataset:** Each compiled binary is converted to an LLVM-IR file using RetDec. This results in an LLVM-IR dataset with four subsets, again corresponding to each of the four compilation configurations.

This structured approach produces two distinct datasets – assembly and LLVM-IR –each organized into four subsets by architecture and compiler configuration. This setup facilitates comparative analysis across different representations, allowing for in-depth performance evaluation and insights specific to each compilation and lifting configuration.

Data Preprocessing: Each test case underwent preprocessing tailored to its structure. This stage included normalization and function extraction, ensuring compatibility with the transformer model input requirements. Additional steps such as token-length limitation and de-duplication were applied to reduce redundancy, enhancing the model's ability to generalize across diverse binary patterns. Data balancing was then used to standardize the dataset size across configurations, supporting unbiased model evaluation.

Model Training and Fine-tuning: Separate CodeBERT transformer models were fine-tuned for each representation language. Fine-tuning involved training assembly and LLVM-IR models independently on their respective datasets to capture unique language characteristics. The dataset partitioning varies for the different experiment levels. For the Basic Level Experiment, the dataset was split into 70% training, 15% validation, and 15% testing. In the Medium Level Experiment, we used an 80-20 training-validation split, incorporating a 5% sample from the test configuration in the training set to simulate real-world exposure. In the Difficult Level Experiment, an 80-20 training-validation split was used, reserving an entire compilation configuration exclusively for testing. JSON formatting was applied to standardize inputs, ensuring consistency across datasets. These varying splits allow for robust model evaluation while preserving independent test sets where applicable. The dataset partitioning is described in greater detail in Section 6.1.

Evaluation: After fine-tuning, the models were tested on an unseen test set, also formatted in JSON, to assess performance in vulnerability classification. Each function was classified as either "vulnerable" or "not-vulnerable" and further categorized by vulnerability type where applicable. The performance metrics of accuracy, precision, F1-score, and recall were used to measure detection effectiveness across representation languages and compilation settings, with results providing insights into the relative robustness of assembly and LLVM-IR for binary vulnerability detection.

4 Dataset Preparation

To evaluate and test our models for vulnerability detection, we required a labeled dataset of compilable source code or binaries tagged with specific Common Weakness Enumeration (CWE) types. CWE is a standardized classification system developed by MITRE to systematically categorize software and hardware security weaknesses [17]. This taxonomy, which helps in identifying and addressing vulnerabilities consistently across different security frameworks, enabled us to assign labels suitable for training and testing.

In this study, we began preprocessing by converting the compiled Juliet Test Suite [23] binary files into LLVM-IR and assembly representations. Then, we standardized, cleaned, and extracted relevant functions, removing those exceeding 800 tokens as outlined in Section 5.2.

Duplicates were removed to enhance data quality, and the dataset was balanced, preparing it for the training phase as described in Section 4.3.

As part of our dataset preparation, we made a minor modification to the `io.c` support file based on [5]. This file contains 18 placeholder functions (`good1` through `good9` and `bad1` through `bad9`), which are empty and appear in every test case after linking. To prevent confusion during analysis, these functions were renamed to avoid any mistaken association with actual examples in the test suite.

4.1 Dataset Creation

In our evaluation, we utilized the Juliet Test Suite v1.3 for C/C++, a widely used benchmark for vulnerability detection, sourced from Software Assurance Reference Dataset(SARD) [18] as

organized in the open GitHub repository [23]. This dataset is organized by CWE type, and the test cases indicate whether vulnerabilities are present or absent [24].

To simulate real-world variability, binaries were generated under diverse compilation conditions by manipulating the following parameters:

- **Optimization:** Optimization levels control how much a compiler enhances the efficiency performance, reducing execution time and memory usage of compiled binaries by applying techniques like code inlining and loop unrolling. While these optimizations improve performance, they also complicate vulnerability detection by altering the original code structure [15]. In our research, we used O0, which applies no optimization, to focus on comparing the structural differences between LLVM-IR and assembly without interference from optimization artifacts. This allows a clearer evaluation of how each representation handles the code's fundamental structure and limits the introduction of variability with the use of various optimization techniques.
- **Architecture:** Computer architectures define the hardware design and influence the compilation process by determining the specific instructions generated by the compiler [9]. This is especially evident in assembly, which directly reflects hardware-specific instructions. For this study, we selected two architectures – ARM (32-bit) and x86-64 – to represent a range of hardware platforms, from low-power embedded systems (e.g., ARM) to more complex desktop systems (e.g., x86-64). This diversity allows us to compare how LLVM-IR, which abstracts architecture details, and assembly, which is closely tied to hardware, perform across different architectures and determine which offers greater robustness.
- **Compiler:** Compilers transform source code into machine-readable binaries, and different compilers can produce variations in execution speed, memory usage, and structure due to their unique features and optimization techniques [15, 21, 25]. To introduce variability in our dataset, we used two widely adopted compilers: GCC and Clang. By including both, we ensure that the binaries reflect diverse compilation techniques, allowing us to test the robustness of LLVM-IR and assembly representations across different compilation environments.

This setup produced four unique combinations of compilation parameters (2 architectures $\times$ 2 compilers $\times$ 1 optimization level), resulting in a comprehensive dataset that reflects diverse real-world compilation scenarios. Further details regarding the compilation environment are provided in Section 5.

4.2 Data Preprocessing

We focused on function-level vulnerability detection, where the objective is to determine whether individual functions within the binary files are vulnerable, and if so, classify them according to their CWE type. This approach allowed for more targeted detection, enabling the model to identify vulnerabilities in specific portions of the code rather than analyzing the binary as a whole. Some test cases in the Juliet Test Suite contain terms like "sink," "source," or "helper," indicating the presence of data flow complexities or that the vulnerable part is not directly within the primary bad function; these cases fall outside the scope of this research. As part of the preprocessing step, we extracted and filtered functions from the compiled binaries, ensuring that the dataset includes both vulnerable and non-vulnerable functions. We selected 23 distinct vulnerability types following the approach in [24] (see Table 2), as categorized in [17]. Each function was assigned a label between 0 and 23, where 0 denotes non-vulnerable functions, and 1 through 23 correspond to specific CWE type.

Table 2. Summary of CWE IDs and corresponding titles.

S.No.	CWE ID	Title
1	CWE197	Numeric Truncation Error
2	CWE401	Missing Release of Memory after Effective Lifetime
3	CWE121	Stack-based Buffer Overflow
4	CWE122	Heap-based Buffer Overflow
5	CWE194	Unexpected Sign Extension
6	CWE23	Relative Path Traversal
7	CWE127	Buffer Under-read
8	CWE195	Signed to Unsigned Conversion Error
9	CWE415	Double Free
10	CWE190	Integer Overflow or Wraparound
11	CWE762	Mismatched Memory Management Routines
12	CWE126	Buffer Over-read
13	CWE680	Integer Overflow to Buffer Overflow
14	CWE36	Absolute Path Traversal
15	CWE78	Improper Neutralization of Special Elements used in an OS Command
16	CWE191	Integer Underflow (Wrap or Wraparound)
17	CWE400	Uncontrolled Resource Consumption
18	CWE369	Divide by Zero
19	CWE457	Use of Uninitialized Variable
20	CWE690	Unchecked Return Value to NULL Pointer Dereference
21	CWE134	Use of Externally-Controlled Format String
22	CWE124	Buffer Underwrite (Buffer Underflow)
23	CWE590	Free of Memory Not on the Heap

Preprocessing was conducted by adapting methods from [5, 24] to ensure the data were optimally prepared for model training. These steps include both essential preprocessing methods and modifications made to enhance data consistency, quality, and relevance for our work:

- We found that in our LLVM-IR datasets, labels mainly followed the pattern "label dec_label_pc_NUM", where NUM indicates the label number. We observed instances where "label" was present, but the "dec_label_pc_" pattern was altered. To ensure consistency, we standardized these cases to match the "label dec_label_pc_NUM" format before processing it further.
- The approach in [5] emphasized Intel syntax assembly. To ensure that our methodology remains unbiased towards any specific architecture or compilation setup, we removed the flags enforcing this syntax, resulting in assembly code represented in the default AT&T syntax via objdump.
- The data were cleaned and normalized by renaming local function names, global variables, labels, and stack variables in LLVM-IR to generic identifiers (e.g., FUNC). Most of the ARM LLVM-IR files contained a unique pattern— "@\$...", we decided to keep this as it is. In assembly, we also normalized the local function names and global variables, stack pointer, and frame pointer. This process was implemented to prevent data leakage and maintain consistency across the datasets, ensuring that the model does not rely on specific names that could introduce bias.
- The method used in [24] standardizes numbers by separating them into individual digits. However, in our approach, we opted not to normalize numbers across either representation. Additionally, while [24] uses Word2Vec embeddings, we opted to use functions in text form, omitting the embedding stage from our preprocessing pipeline and directly passing the functions to the transformer tokenizer.

- All comments were removed to reduce data noise. In [5], comments were replaced with the corresponding operand, however, we chose to remove them completely to align with the LLVM-IR approach and focus exclusively on executable instructions. This adjustment allowed the model to concentrate solely on the structural and functional elements of the code necessary for vulnerability detection.
- In our assembly representation, we focused solely on the executed instructions, excluding addresses and binary representation of the command.
- We appended "\n" at the end of each line (except the last line in each function) to indicate the end of a line.
- Primary and secondary good functions, along with primary bad functions, were extracted using regex patterns specified in [24] and [5]. Specifically, we applied the LLVM-IR regex from [24] and the assembly regex from [5].
- The preprocessed data were converted into JSON format, with each function labeled according to one of 24 categories: 0 for non-vulnerable functions and 1-23 for different CWE types.
- After preprocessing, we ensured that no CWE identifiers were left in the data by replacing each instance with "FUNC". This step was important to verify that no predefined patterns were overlooked, ensuring consistency and preventing data leakage into the model.

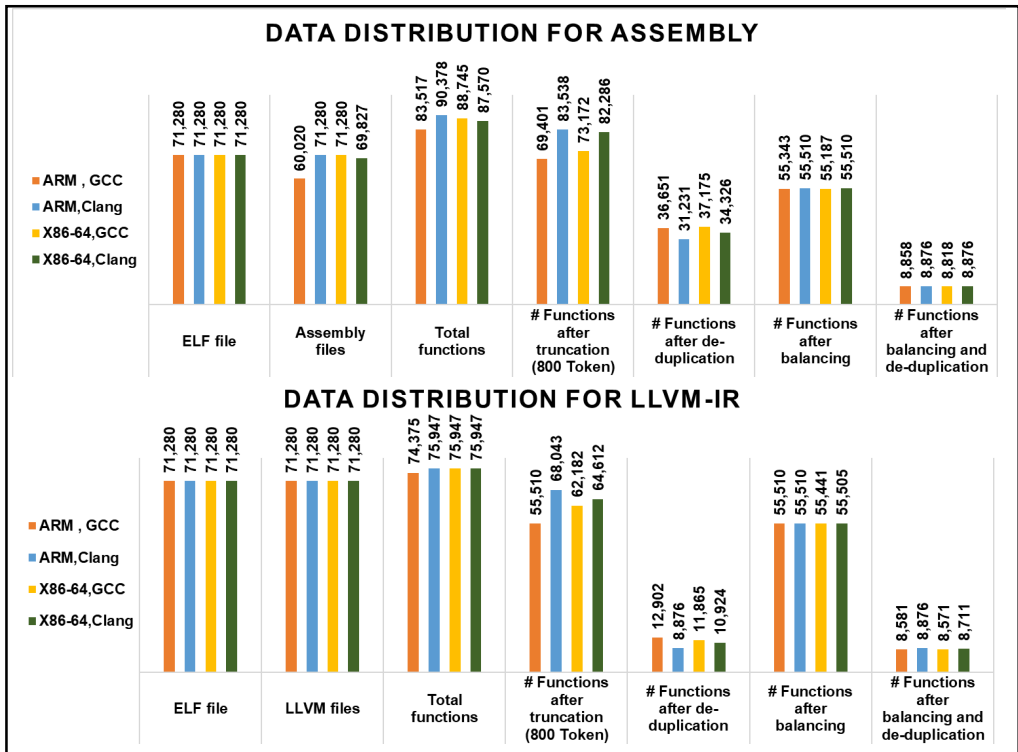


Fig. 2. Data distribution across compilation environments.

Note that some test cases have been omitted during preprocessing due to configuration settings, potentially affecting the data distribution across certain compilation environments.

4.3 Data De-duplication and Balancing

While preparing the data for analysis, we performed test-case de-duplication to prevent overfitting and applied data balancing. This step was necessary to avoid one majority compilation dominating the dataset and leading to overfitting. Such dominance could skew the classification learning process and reduce the model's ability to generalize across different configurations.

4.3.1 Data De-duplication. Despite the inherent uniqueness of the Juliet Test Suite's test cases, duplicates can arise during preprocessing, particularly after the compilation and lifting to different representation stages [5]. Our analysis of the data distribution (presented in Figure 2) revealed that 82.2% of the LLVM-IR dataset and 54.8% of the assembly dataset contained duplicates. To assess the impact of de-duplication on model performance, we conducted a comparative analysis by fine-tuning the model on both the original dataset (containing duplicates and the de-duplicated dataset) as explained in Section 6.

4.3.2 Data Balancing. After truncation, the dataset showed an uneven distribution of functions across compilation configurations as shown in Figure 2. To ensure a fair comparison and confirm that the robustness of the representation language is not influenced by distribution differences, we standardized all datasets to the same size. This was done using proportionate sampling, aligning the dataset with the CWE distribution of the smallest configuration. We accomplished this by randomly selecting functions according to the CWE proportions in the smallest configuration, preventing any single configuration from dominating the training process. This approach ensured that each compilation configuration contributed equally while maintaining the original CWE distribution, avoiding bias toward the most frequent configurations. By implementing this balancing strategy, we validated that our results are free from distribution artifacts and accurately represent the true performance differences between representation languages.

Figure 2 provides an overview of the dataset preparation phase, illustrating how data is processed, from compilation to final function selection. The process begins with 71,280 ELF files across all compilation configurations. After lifting them to assembly and LLVM-IR, the total number of extracted functions varies, reaching up to 90,378 for assembly and 75,947 for LLVM-IR. When applying a truncation step to remove functions exceeding 800-tokens, the number of functions is reduced to between 69,401 and 83,538 for assembly and between 55,510 and 68,043 for LLVM-IR. Further processing is performed using two approaches: (1) balancing without removing duplicates retains approximately around 55,000 functions in assembly and LLVM-IR, while (2) removing duplicates and performing balancing further reduces the dataset to approximately 8,000 functions for both assembly and LLVM-IR.

5 Experimental Setup

The experiments in this research were conducted on a Linux x86-64 environment using four compilation configurations. Two configurations were based on the x86-64 architecture, compiled with GCC and Clang. To introduce additional variability, we cross-compiled the Juliet Test Suite targeting the ARM 32-bit architecture, again using both GCC and Clang as compilers. We used ld.lld as the linker with Clang for both x86-64 and ARM, and used arm-linux-gnueabi-hf-ld as the linker when cross-compiling ARM with GCC. To maintain consistency, we used llvm-objdump for files compiled with Clang, arm-linux-gnueabi-hf-objdump for ARM files compiled with GCC, and the standard objdump for x86-64 files compiled with GCC. In ARM cross-compilation with Clang, we set position independent code (PIC) to 'false' as instructed in the documentation [14]. By doing that we generated four unique compilation setups across multiple architectures and compilers to

create four different ELF datasets. When executed simultaneously, the compilation process took approximately three hours. Additionally, the model fine-tuning was conducted on a GPU-enabled system.

5.1 Model Architecture and Training

CodeBERT-Based Classification Model: In this research, we employed CodeBERT-base, a pre-trained transformer-based model designed for processing source code and natural language. We obtained the model from Hugging Face's model repository at [31]. We decided to use CodeBERT due to its ability to capture contextual relationships and complex dependencies within code sequences, making it highly effective for vulnerability detection tasks.

We fine-tuned CodeBERT separately on both the LLVM-IR and assembly datasets, which contained functions labeled as the 23 CWE types and the 'not vulnerable' category previously described. This fine-tuning allowed the model to specialize in detecting vulnerabilities in each representation while providing insights regarding the robustness of LLVM-IR and assembly under varied compilation settings.

Training Configuration: The LLVM-IR and assembly datasets were trained using the same configurations as described in Figure 1.

Our training parameters were based on those in [5], with a batch size of 16 samples per GPU, a learning rate of $1.1e-5$, and a weight decay of $3e-4$. The model was trained for 30 epochs, with early stopping applied if no significant improvements were observed after three epochs. In our datasets, the average sequence length is below CodeBERT's 512-token limit, allowing us to efficiently utilize the model.

Performance Metrics: To assess the effectiveness of LLVM-IR and assembly representations, the model's performance in multi-class vulnerability detection was evaluated using the performance metrics of accuracy, precision, recall, and F1-score. Accuracy reflects the percentage of correct classifications, while precision and recall measure the model's success in identifying vulnerabilities accurately without misclassifying non-vulnerable functions. The F1-Score provides a balanced metric across CWE categories.

5.2 Research Challenges and Limitations

Handling Compilation Variability: Ensuring consistent model performance across a wide range of compilation configurations that include different architectures (ARM, x86-64), compilers (GCC, Clang), and optimization level (O0) was a key challenge. Compilation variability introduces subtle changes in the code structure that can influence the model's ability to detect vulnerabilities. To handle this, we adopted a multi-task learning approach, enabling CodeBERT to learn from multiple configurations simultaneously. We also curated the dataset, a step that included data balancing and de-duplication, to ensure that no single compilation configuration dominated the learning process; this enhanced the model's generalizability.

Bit Version: In this study, we retained the i32 and i64 distinctions in LLVM IR, as they reflect the bit version (32-bit vs. 64-bit) rather than indicating vulnerabilities. This aligns with assembly code where such distinctions are less explicit, ensuring consistency across representations and allowing the model to focus on meaningful vulnerability patterns rather than superficial architectural details.

Function Length Selection: To ensure that the model learns from meaningful code sequences, we applied a function length constraint, selecting only functions with fewer than 800-tokens. This

Table 3. Comparison of Longformer and CodeBERT models performance on LLVM-IR and assembly datasets following balancing methods (22 labels).

Language	Model	#Test Cases	Token Length Limitation	Token Limitation in the Dataset	Runtime Duration	Accuracy
LLVM-IR	Longformer-base-4096	17,734	1500	-	6h 1m	93.2%
	CodeBERT	17,734	512	-	0h 40m	89.6%
	CodeBERT	18,571	512	512	0h 49m	95.0%
	CodeBERT	17,694	512	800	0h 42m	91.0%
Assembly	Longformer-base-4096	17,734	1500	-	7h 35m	88.1%
	CodeBERT	17,734	512	-	0h 45m	85.8%
	CodeBERT	18,571	512	512	0h 37m	89.3%
	CodeBERT	17,694	512	800	0h 40m	87.6%

Table 4. Comparison of Longformer and CodeBERT models performance on LLVM-IR and assembly datasets with different token length limitations (24 labels).

Language	Model	#Test Cases	Token Length Limitation	Token Limitation in the Dataset	Runtime Duration	Accuracy
LLVM-IR	Longformer-base-4096	48,487	1500	-	19h 34m	97.3%
	CodeBERT	48,487	512	-	1h 30m	91.7%
	CodeBERT	18,571	512	512	0h 50m	95.3%
	CodeBERT	34,739	512	800	1h 15m	94.2%
Assembly	Longformer-base-4096	48,488	1500	-	19h 24m	92.1%
	CodeBERT	48,488	512	-	1h 31m	88.4%
	CodeBERT	19,285	512	512	0h 46m	90.1%
	CodeBERT	35,428	512	800	1h 18m	89.8%

decision was based on several key factors. Capping the function length at 800-tokens prevents excessive truncation and helps retain essential function structures. This threshold also strikes a balance between preserving detailed code patterns and maintaining computational efficiency, enabling CodeBERT to effectively detect vulnerabilities while adhering to the 512-token limit through truncation. Additionally, excluding very long functions reduces the risk of overfitting, improving the model's ability to generalize to diverse real-world scenarios. To establish an optimal function length threshold, we conducted an empirical analysis before training, analyzing function token distributions across assembly and LLVM-IR. Our findings indicated that only 14.359% of functions exceeded 800-tokens, making it as a well-balanced choice between function length and computational feasibility. This threshold was chosen to align with the constraints of transformer models such as CodeBERT, ensuring efficient training while preserving meaningful vulnerability patterns. To further validate this decision, we conducted an additional experiment (Tables 3,4) using Longformer-base-4096[10]. This model is designed to process longer sequences, making it suitable for evaluating the impact of token length limitations. The experiment compared different CodeBERT configurations to assess how these limitations affect performance. Two evaluation approaches were used: a standalone evaluation approach, where datasets were tested independently with varying dataset sizes and CWE label coverage (all models were fine-tuned with the number of labels set at 24 for comparison purposes), and a balanced comparison approach (base on Section 4.3.2) which standardized token distributions by aligning with the strictest constraint (CodeBERT, with a 512-token limit), retaining 22 of the 24 CWE labels. Our results showed that LLVM-IR consistently outperformed assembly, highlighting its robustness for vulnerability detection. Applying a strict 512-token cutoff led to high accuracy (95.3%, as seen in Table 4), but it significantly reduced the dataset size and eliminated two CWE labels, potentially limiting the model's generalization capability. On the other hand, using Longformer, which can process longer sequences, achieved even higher accuracy (97.3%), but its runtime (19h 34m) makes it impractical for large-scale deployment. In contrast, the 800-token limit allowed CodeBERT to maintain strong accuracy (94.2%) while keeping training runtime reasonable (1h 15m), highlighting its ability to

optimally manage the trade-off between efficiency and dataset completeness.

Research Limitation: Given the inconsistency in vulnerability representation across different compilation configurations, which results from variability in the lifting processes or compiler optimizations, certain vulnerabilities may not be fully represented. With a reliance on tools like objdump and RetDec [3], the consistency and completeness of data may also be impacted by tool-specific design or platform constraints, resulting in the possible omission of certain vulnerabilities across some representations.

6 Evaluation and Results

In this research, our experiments aimed to answer the following research questions:

- **RQ1: Which representation language (assembly or LLVM-IR) is more robust for detecting vulnerabilities in binary files when the compilation process is unknown?**
- **RQ2: Does using a transformer as a classification model improve the accuracy of vulnerability detection in binary files in the LLVM-IR domain?**

6.1 Robustness Evaluation: LLVM-IR vs. Assembly

To address *RQ1*, we designed and conducted three experiments with varying levels of difficulty for each representation separately. This range of experimental setups allowed us to derive insights from multiple perspectives.

- **Basic Level Experiment - All Combinations Included:** In this experiment, we included all four compilation configurations and split the entire dataset with a 70-15-15 ratio to create the training, validation, and test sets respectively.
- **Medium Level Experiment - Partial Exposure:** In this experiment, we used three out of the four compilation configurations for training and validation, incorporating a small sample from the test configuration into the training set to mimic real-world variability. Specifically, 5% of the data from the test configuration was included in the training set, while the remaining 95% was reserved for testing. This setup enabled us to evaluate the model's generalization ability across diverse compilation settings, simulating scenarios where only limited data from certain configurations is available for learning. In this experiment, we divided the training data set into 80% for training and 20% for validation.
- **Difficult Level Experiment - Leave-One-Out:** we used three out of four compilation configurations for training and validation, reserving the fourth configuration exclusively for testing. This setup enabled us to assess how well each representation is generalized across diverse compilation conditions, providing insights into the robustness of assembly and LLVM-IR representations when one compilation is unknown. In this experiment, we divided the training data set into 80% for training and 20% for validation.

Test Setup: For each experiment, we designed a specific test setup. In the basic and medium levels of experiments, we used only the prepared balanced de-duplicated data, as retaining duplicates could lead to overlapping entries between the training and testing sets, which would risk data leakage and distort the results. For further validation, each of these two experiments was repeated using two different dataset splits derived from our data. This approach ensured that the results were consistent and not dependent on any specific split configuration.

For the difficult level experiment, we expanded the setup to assess the model's sensitivity to duplicate data. We trained the model twice, once on de-duplicated data and once with the duplicates

Table 5. Experimental results: basic level experiment - all combination included

Exp.	Split ID	Duplicates	Representation	Accuracy	Precision	Recall	F1-Score	# Train Functions	# Validation Functions	# Test Functions
Basic	Split1	Removed	LLVM-IR	93.6%	93.8%	93.6%	93.6%	24,317	5,211	5,211
			Assembly	91.0%	91.0%	91.0%	90.7%	24,799	5,315	5,314
	Split2	Removed	LLVM-IR	94.2%	94.2%	94.2%	94.1%	24,317	5,211	5,211
			Assembly	89.8%	89.1%	89.8%	89.1%	24,799	5,315	5,314

included. In both cases, the model was evaluated on a de-duplicated test set, enabling a direct comparison of how duplicates in training data impact model performance. This approach allowed us to determine whether duplicates impact the robustness of vulnerability detection across varied compilation settings.

Table 6. Experimental results: medium level experiment - partial exposure

Exp.	Split ID	Duplicates	Representation	Compilation	Accuracy	Precision	Recall	F1-Score	# Train Functions	# Validation Functions	# Test Functions
Medium	Split1	Removed	LLVM-IR	OO, ARM, Clang	77.9%	76.9%	78.0%	76.5%	21,045	5,262	8,432
				OO, ARM, GCC	85.2%	86.3%	85.2%	85.0%	21,270	5,318	8,151
				OO, x86-64, Clang	89.4%	89.6%	89.4%	89.1%	21,171	5,293	8,275
				OO, x86-64, GCC	90.6%	90.8%	90.6%	89.6%	21,277	5,320	8,142
		Assembly	LLVM-IR	OO, ARM, Clang	73.8%	71.2%	73.8%	70.9%	21,596	5,400	8,432
				OO, ARM, GCC	75.9%	73.4%	75.9%	73.5%	21,610	5,403	8,415
				OO, x86-64, Clang	82.5%	82.5%	82.5%	81.9%	21,596	5,400	8,432
				OO, x86-64, GCC	77.3%	74.4%	77.3%	74.9%	21,640	5,411	8,377
	Split2	Removed	LLVM-IR	OO, ARM, Clang	80.0%	78.9%	80.0%	78.2%	21,045	5,262	8,432
				OO, ARM, GCC	86.6%	87.3%	86.6%	86.5%	21,270	5,318	8,151
				OO, x86-64, Clang	89.7%	89.9%	89.7%	89.4%	21,171	5,293	8,275
				OO, x86-64, GCC	91.2%	91.1%	91.2%	90.8%	21,277	5,320	8,142
		Assembly	LLVM-IR	OO, ARM, Clang	70.3%	70.2%	70.3%	69.9%	21,596	5,400	8,432
				OO, ARM, GCC	75.4%	73.0%	75.4%	72.9%	21,610	5,403	8,415
				OO, x86-64, Clang	80.0%	79.8%	80.0%	79.3%	21,596	5,400	8,432
				OO, x86-64, GCC	78.8%	76.3%	78.8%	76.2%	21,640	5,411	8,377

Table 7. Experimental results: difficult level experiment - leave-one-out

Exp.	Duplicates	Representation	Compilation	Accuracy	Precision	Recall	F1-Score	# Train Functions	# Validation Functions	# Test Functions
Difficult	Remain	LLVM-IR	OO, ARM, Clang	79.5%	77.7%	79.5%	76.5%	133,164	33,292	55,510
			OO, ARM, GCC	83.6%	86.3%	83.6%	84.4%	133,164	33,292	55,510
			OO, x86-64, Clang	89.7%	91.8%	89.7%	90.3%	133,168	33,293	55,505
			OO, x86-64, GCC	89.0%	90.2%	89.0%	89.2%	133,220	33,305	55,441
		Assembly	OO, ARM, Clang	74.8%	68.6%	74.8%	70.5%	132,832	33,208	55,510
			OO, ARM, GCC	78.8%	78.8%	75.4%	75.4%	132,965	33,242	55,343
			OO, x86-64, Clang	77.4%	70.2%	77.4%	70.9%	132,832	33,208	55,510
			OO, x86-64, GCC	60.6%	69.8%	60.6%	64.4%	133,090	33,273	55,187
	Removed	LLVM-IR	OO, ARM, Clang	72.9%	66.6%	72.9%	67.4%	20,690	5,173	8,876
			OO, ARM, GCC	72.3%	79.6%	72.3%	74.7%	20,926	5,232	8,581
			OO, x86-64, Clang	86.8%	87.3%	86.8%	86.6%	20,822	5,206	8,711
			OO, x86-64, GCC	83.8%	83.7%	83.8%	81.5%	20,934	5,234	8,571
		Assembly	OO, ARM, Clang	54.9%	56.1%	54.9%	55.1%	21,241	5,311	8,876
			OO, ARM, GCC	63.7%	61.1%	63.7%	57.5%	21,256	5,314	8,858
			OO, x86-64, Clang	67.0%	62.6%	67.0%	59.4%	21,241	5,311	8,876
			OO, x86-64, GCC	52.3%	59.9%	52.3%	54.3%	21,288	5,322	8,818

As can be seen in the results presented in Tables 5- 7, the performance of LLVM-IR is superior to that of assembly in detecting vulnerabilities in binary files, especially in scenarios where the compilation parameters are unknown. The following insights can be derived from these findings:

- **Basic Level Experiment:** In this setup, data from all compilation configurations were included in the training process. The LLVM-IR representation excelled, achieving accuracies of 93.6% and 94.2% across the two splits. While assembly also performed well, its accuracy was lower, with scores of 91% and 89.8% for the two splits respectively. A performance gap of 2-3% between LLVM-IR and assembly was observed across all metrics in the first split, while a larger 4-5% gap appeared in the second split, highlighting LLVM-IR's adaptability and robustness in multi-class vulnerability detection. This finding suggests that LLVM-IR

provides a more consistent approach, particularly for detecting a range of vulnerability types across varied data splits.

- **Medium Level Experiment:** In this setup, the results show a decline in performance metrics across most configurations; this is not unexpected given that 95% of one compilation was excluded from the training set. Despite this constraint, LLVM-IR maintained accuracy of over 80% across most configurations, with strong precision (up to 91.1%) and recall (up to 91.2%), demonstrating its robustness. In contrast, assembly exhibited greater fluctuation, with accuracy dropping as low as 70.3% in certain configurations (ARM with Clang). These results suggest that LLVM-IR provides greater robustness for cross-compilation generalization, maintaining relatively high-performance metrics even with limited exposure to specific configurations.
- **Difficult Level Experiment:** In this setup, one compilation configuration was entirely excluded from training, allowing us to assess the model's adaptability on unseen compilations. With duplicates, accuracy for LLVM-IR remained stable across configurations, with results like 83.6% for ARM with GCC and 89.7% for x86-64 with Clang. However, removing duplicates led to a decline in accuracy, especially in assembly, where accuracy dropped significantly from 74.8% (ARM with Clang) with duplicates to 54.9% without. This pattern suggests that duplicates introduce a level of familiarity that may artificially boost model performance by reinforcing certain patterns learned during training. Despite the drop in accuracy, removing duplicates is essential to ensure authentic generalization and prevent overfitting, as seen in real-world scenarios where functions rarely appear identically across multiple compilations. Notably, LLVM-IR maintained better stability without duplicates, achieving 86.8% accuracy for x86-64 with GCC, compared to the assembly's lower and more variable performance.

Our results consistently demonstrate that LLVM-IR outperforms assembly in vulnerability detection under multi-compilation scenarios where compilation parameters are unknown. This superiority derives from LLVM-IR's architecture-agnostic nature and structural stability. While assembly provides a fine-grained perspective on hardware interactions, this advantage is offset by its sensitivity to compiler configurations and instruction set variations. Even minor changes in compiler versions or target architectures can significantly alter the instruction set, register allocation, and memory layout, leading to inconsistencies that make pattern recognition for machine learning models harder to understand. In contrast, LLVM-IR abstracts away some of these variations and maintains a more consistent structure across different compilation settings. This consistency makes it more reliable for vulnerability detection in real-world scenarios where compilation details are unknown. Beyond its stability, LLVM-IR also provides richer representations that enhance transformer-based model performance. Assembly instructions are often short and fragmented, making it harder to capture patterns between different compilation configurations. LLVM-IR, in contrast, includes explicit type information, structured control flow, and higher-level abstractions, allowing the model to better capture contextual relationships and maintain coherence across compilations. Our leave-one-out experiments confirm this advantage by showing that LLVM-IR has smaller performance drops than assembly, demonstrating its better generalization to unseen compilations. Additionally, our results indicate that LLVM-IR's accuracy remains more stable across different compiler configurations, whereas assembly exhibits significant variability, particularly when trained on limited compilation settings. As expected, accuracy declined as test complexity increased, reflecting the unique challenges introduced by varying compilation configurations. However, LLVM-IR remained more stable, reinforcing the argument that it is better suited for scenarios requiring model adaptability across diverse compilation settings. Furthermore, our experiments revealed that certain vulnerability

types, such as CWE36 and CWE23, became increasingly difficult to detect as test complexity grew, likely due to their reduced representation in the dataset.

6.2 Evaluation of CodeBERT Transformer's Effectiveness

6.2.1 Evaluating CodeBERT for Vulnerability Classification in LLVM-IR With a Single Compilation.

To evaluate the effectiveness of transformers for vulnerability classification with LLVM-IR, we compared our results against those reported in [24]. That study utilized GRU, LSTM, and SRNN models for binary and multi-class vulnerability detection, achieving 88% accuracy for binary classification and 77% for multi-class.

We used and compiled the same dataset from [23] with a modified preprocessing pipeline, omitting the word2vec embeddings to align with our approach of directly using a transformer tokenizer. To assess the impact of token length, we evaluated our model's performance under two conditions: one limiting functions to 1,000 tokens (as described in [24]) and another without token limitation. Binary classification accuracy remained stable, with 88.4% accuracy achieved with the token limit and 87% without, indicating consistent performance in detecting vulnerabilities. However, in multi-class classification, our transformer model showed a notable improvement, reaching 86.6% accuracy with the token limit and 85.9% without an approximately 10% increase over previous work [24]. The performance boost suggests that the transformer model's ability to capture long-range dependencies and complex patterns in code representations contributes significantly to the improved accuracy, contributing to its superior performance in distinguishing between vulnerable and non-vulnerable functions.

Table 8. Comparison of RNN, LSTM, and GRU models performance for vulnerability detection using Word2vec embeddings. The dataset was preprocessed with an 800 token limit, and each input sequence was padded or truncated to a fixed length of 1000 words in the Word2vec representation.

Language	Classification Task	# of Layers	Accuracy
RNN	Unidirectional	1	71.27%
		2	72.50%
	Bidirectional	1	72.39%
		2	77.28%
		3	77.39%
		3 (unit size - 128)	80.85%
LSTM	Unidirectional	1	68.59%
		2	68.59%
	Bidirectional	1	71.50%
		2	76.63%
GRU	Unidirectional	1	72.14%
		2	81.04%
	Bidirectional	1	71.98%
		2	84.13%

6.2.2 Comparing CodeBERT Transformer-Based Classification in LLVM-IR with RNN Approaches in Multi-Compilation Environments.

As shown in Table 1, the majority of the models primarily belong to the RNN family or rely on models belonging to the RNN family, including RNN, LSTM, and GRU. To compare the effectiveness of transformers in a multi-compilation environment to state-of-the-art methods, we performed a comprehensive evaluation of our transformer model and RNN-based approaches, employing two embedding strategies with the RNNs:

- (1) **Roberta-based embedding:** We applied the RobertaTokenizer (CodeBERT-base) following our methodology and embed the text using RobertaModel before feeding it into various RNN models.

Table 9. Comparison of RNN, LSTM, and GRU models performance for vulnerability detection using RobertaTokenizer (CodeBERT-base) embeddings. The dataset was preprocessed with an 800-token limit, and each input sequence was padded or truncated to meet BERT'S 512-token limit

Language	Classification Task	# of Layers	Accuracy
RNN	Unidirectional	1	71.52%
		2	73.88%
	Bidirectional	1	78.26%
		2	79.60%
		3	79.16%
		3 (unit size - 128)	79.75%
LSTM	Unidirectional	1	81.71%
		2	78.35%
	Bidirectional	1	85.93%
		2	81.44%
GRU	Unidirectional	1	82.69%
		2	83.19%
	Bidirectional	1	83.61%
		2	84.09%

- (2) **Using Word2vec:** We employed Word2Vec, a widely recognized embedding technique in the vulnerability detection domain, to tokenize the dataset before feeding it into the RNN models.

Given the need for a reliable baseline architecture, we selected one that has demonstrated strong performance in the LLVM-IR domain. This choice allowed us to compare the capabilities of different models based on a well-established and proven architecture. We adopted the same approach as in [24]. Since the original implementations from that paper were not available, we carefully reproduced RNN, LSTM, and GRU models based on the methodologies described in that study.

We performed this evaluation on the data used in the *"Basic Level Experiment"* - split 2 described in Section 6.1 and compare the results obtained to the results of that experiment. As shown in Table 8, 9 the overall performance was similar for the two embedding strategies, with a slight improvement observed when using RobertaModel (CodeBERT-base) for embedding. This outcome was in line with our expectations, as CodeBERT is specifically trained on code-related data, making it more effective in capturing contextual representations relevant to vulnerability detection than a general-purpose embedding like Word2vec. While RNN-based models achieved reasonable classification accuracy, our transformer model achieved a remarkable 94.2% accuracy, significantly surpassing the highest result obtained with a bidirectional LSTM with CodeBERT embeddings (85.93%). This improvement over existing static analysis methods, demonstrates the effectiveness of transformers in multi-compilation settings.

6.2.3 Evaluating CodeBERT's Performance in LLVM-IR on Unseen Optimization Level. In addition, to support the primary goal of our study we have now expanded our evaluation to include -O2 optimization, further assessing the model's robustness under additional compilation settings. We perform a binary classification task, where any non-zero CWE label was labeled as "1" (vulnerable). To ensure the reliability of our evaluation, we implemented a pre-filtering phase to exclude functions that had undergone optimization and no longer accurately reflected the presence or absence of vulnerability. We used GPT-4o-mini [19] with the temperature parameter set to zero, ensuring that the model produced consistent outputs for identical inputs. After performing the filtering process and achieving a balance between vulnerable and non-vulnerable samples, we constructed a test case, evaluating 148 functions, covering 14 of the 24 labels. We then fine-tuned our "Basic Level

Experiment" Split 2 model on a relevant subset of test cases containing the remaining labels after -O2 filtering, treating all labels as binary (0 for non-vulnerable, 1 for vulnerable). In our evaluation, we performed an inference-only test using the O2-filtered dataset to assess the model's ability to detect vulnerabilities in an unseen dataset. The fine-tuned model achieved 76.4% accuracy, demonstrating that the model remains effective in identifying vulnerabilities even after optimization. This result aligns with the unseen evaluation outcome reported "Difficult Level Experiment - Leave-One-Out" in Section 6.1.

7 Conclusion And Future Work

In this research, we evaluated the robustness of LLVM-IR and assembly in vulnerability detection across varying compilation configurations and examined the potential of the CodeBERT transformer-based model for enhancing detection performance within the LLVM-IR domain. The results of our experiments offer insights into the strengths and limitations of these representation languages in conditions reflecting the real-world variability in compilation parameters. A key finding was LLVM-IR's stability in handling diverse architectures and compilers, outperforming assembly in terms of accuracy. Assembly, while beneficial in capturing hardware-specific details, exhibited sensitivity to changes in compilation settings, resulting in more significant fluctuations between different experiments.

By using a transformer model with LLVM-IR, we observed significant improvements in multi-class detection accuracy compared to RNNs as a classification tool. The transformer's capacity to capture complex, long-range dependencies contributed to higher accuracy, emphasizing its strength in detecting vulnerabilities in LLVM-IR.

Our evaluation demonstrated LLVM-IR's strong potential for unknown compilation scenarios, opening several avenues for future research. For instance, future work could broaden the scope to include additional CWE types beyond the 24 labels currently used (23 CWEs and one non-vulnerable label). Moreover, expanding the analysis to various optimization levels beyond O0 could reveal further insights into LLVM-IR's robustness across a range of optimized compilations, potentially enhancing vulnerability detection capabilities.

The findings from this study highlight LLVM-IR's robustness over assembly for detecting vulnerabilities in binary files when compilation parameters are unknown. Its consistent performance across varied compilation settings highlights LLVM-IR as a reliable representation language, offering greater stability and adaptability for vulnerability detection. Additionally, using a transformer model with LLVM-IR provides a further boost in accuracy by effectively capturing complex code patterns, though the strength of LLVM-IR itself remains the key factor in achieving reliable results.

8 Data Availability

The dataset and source code used in this study are available at https://github.com/rvdac/rvdac-LLVMIR_Assembly

References

- [1] Ashish Adhikari and Prasad Kulkarni. 2025. Survey of Techniques to Detect Common Weaknesses in Program Binaries. *Cyber Security and Applications* 3 (2025), 100061. <https://doi.org/10.1016/j.csa.2024.100061>
- [2] Amy Aumpansub and Zhen Huang. 2022. Learning-Based Vulnerability Detection in Binary Code. In *2022 14th International Conference on Machine Learning and Computing (ICMLC)*. IEEE, Guangzhou, China, 266–271.
- [3] Avast Software. 2024. RetDec: Retargetable Decompiler. <https://github.com/avast/retdec>. Accessed: 2024-10-27.
- [4] Larissa Braz and Alberto Bacchelli. 2022. Software Security During Modern Code Review: The Developer's Perspective. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*. ACM, Singapore, 810–821.

- [5] Clemens-Alexander Brust, Tim Sonneckal, and Bernd Gruner. 2023. ROMEO: A binary vulnerability detection dataset for exploring Juliet through the lens of assembly language. *Computers & Security* 128 (2023), 103165.
- [6] Yaniv David, Nimrod Partush, and Eran Yahav. 2016. Statistical similarity of binaries. *Acm sigplan notices* 51, 6 (2016), 266–280.
- [7] Yaniv David, Nimrod Partush, and Eran Yahav. 2018. Firmup: Precise static detection of common vulnerabilities in firmware. *ACM SIGPLAN Notices* 53, 2 (2018), 392–404.
- [8] Diwan, Ashita and Li, Miles Q and Fung, Benjamin CM. 2022. VGraph2Vec: Vulnerability Detection in Assembly Code using Message Passing Neural Networks. In *2022 21st IEEE International Conference on Machine Learning and Applications (ICMLA)*. IEEE, 1039–1046.
- [9] Michael Flynn. 2007. Computer Architecture. In *Wiley Encyclopedia of Computer Science and Engineering*. John Wiley & Sons, Ltd, Hoboken, NJ, USA. <https://doi.org/10.1002/9780470050118.ecse071>
- [10] Iz Beltagy and Matthew E. Peters and Arman Cohan. 2020. Longformer: The Long-Document Transformer. *arXiv:2004.05150* (2020).
- [11] Litao Li, Steven HH Ding, Yuan Tian, Benjamin CM Fung, Philippe Charland, Weihang Ou, Leo Song, and Congwei Chen. 2023. VulANalyzeR: Explainable Binary Vulnerability Detection with Multi-task Learning and Attentional Graph Convolution. *ACM Transactions on Privacy and Security* 26, 3 (2023), 1–25.
- [12] Munan Li, Hongbo Liu, Xiangdong Jiang, Zheng Zhao, and Tianhao Zhang. 2023. SENSE: An unsupervised semantic learning model for cross-platform vulnerability search. *Computers & Security* 135 (2023), 103500.
- [13] Hongliang Liang, Zhuosi Xie, Yixiu Chen, Hua Ning, and Jianli Wang. 2020. FIT: Inspect vulnerabilities in cross-architecture firmware by deep learning and bipartite matching. *Computers & Security* 99 (2020), 102032.
- [14] LLVM Project. 2024. Arm Cross Compilation With Clang. <https://llvm.org/docs/HowToCrossCompileLLVM.html>. Accessed: 2024-10-27.
- [15] Rahim Mammadli, Marija Selakovic, Felix Wolf, and Michael Pradel. 2021. Learning to make compiler optimizations more effective. In *Proceedings of the 5th ACM SIGPLAN International Symposium on Machine Programming*. ACM, Palo Alto, CA, USA, 9–20.
- [16] Gary A McCully, John D Hastings, Shengjie Xu, and Adam Fortier. 2024. Bi-directional transformers vs. word2vec: Discovering vulnerabilities in lifted compiled code. In *2024 Cyber Awareness and Research Symposium (CARS)*. IEEE, 1–8.
- [17] MITRE Corporation 2024. *Common Weakness Enumeration (CWE)*. MITRE Corporation. <https://cwe.mitre.org/> Accessed: March 22, 2024.
- [18] NIST. 2024. SARD: Software Assurance Reference Dataset. <https://samate.nist.gov/SARD/testsuite.php>. Accessed: 2024-10-27.
- [19] OpenAI. 2024. GPT-4o mini: advancing cost-efficient intelligence. <https://openai.com/index/gpt-4o-mini-advancing-cost-efficient-intelligence/>. Accessed: 2025-02-23.
- [20] OWASP Foundation. 2021. Software and Data Integrity Failures - OWASP Top Ten 2021. https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/. Accessed: 2024-10-30.
- [21] Lucas Palazzi, Guanpeng Li, Bo Fang, and Karthik Pattabiraman. 2019. A Tale of Two Injectors: End-to-End Comparison of IR-Level and Assembly-Level Fault Injection. In *2019 IEEE 30th International Symposium on Software Reliability Engineering (ISSRE)*. IEEE, Berlin, Germany, 151–162.
- [22] Tao Peng, Shixu Chen, Fei Zhu, Junwei Tang, Junping Liu, and Xinrong Hu. 2023. PTLVD: Program Slicing and Transformer-Based Line-Level Vulnerability Detection System. In *Proceedings of the 23rd IEEE International Working Conference on Source Code Analysis and Manipulation (SCAM)*. IEEE, 162–173.
- [23] Alex Richardson. 2024. Juliet Test Suite for C/C++. <https://github.com/arichardson/juliet-test-suite-c>. Accessed: 2024-10-28.
- [24] Andreas Schaad and Dominik Binder. 2023. Deep-Learning-Based Vulnerability Detection in Binary Executables. In *Foundations and Practice of Security*. Springer Nature Switzerland, Cham, Switzerland, 453–460. https://doi.org/10.1007/978-3-031-30122-3_28
- [25] Adam Spanier and William Mahoney. 2023. Static Vulnerability Analysis Using Intermediate Representations: A Literature Review. In *Proceedings of the European Conference on Cyber Warfare and Security*, Vol. 22. Academic Conferences International Limited, Athens, Greece, 458–465.
- [26] Junfeng Tian, Wenjing Xing, and Zhen Li. 2020. BVDetector: A program slice-based binary code vulnerability intelligent detection system. *Information and Software Technology* 123 (2020), 106289.
- [27] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is All You Need. *Advances in Neural Information Processing Systems* 30 (2017), 5998–6008.
- [28] Guangli Wu and Huili Tang. 2023. Binary Code Vulnerability Detection Based on Multi-Level Feature Fusion. *IEEE Access* 11 (2023), 63904–63915. <https://doi.org/10.1109/ACCESS.2023.3289001>

- [29] Han Yan, Senlin Luo, Limin Pan, and Yifei Zhang. 2021. HAN-BSVD: a hierarchical attention network for binary software vulnerability detection. *Computers & Security* 108 (2021), 102286.
- [30] Bin Zeng. 2012. *Static analysis on binary code*. Technical Report. Tech. rep. Tech-report, 2012 (cit. on pp. 17, 22, 23).
- [31] Zhangyin Feng and Daya Guo and Duyu Tang and Nan Duan and Xiaocheng Feng and Ming Gong and Linjun Shou and Bing Qin and Ting Liu and Daxin Jiang and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. [arXiv:2002.08155](https://arxiv.org/abs/2002.08155) [cs.CL]
- [32] Jianyun Zheng, Jianmin Pang, Xiaochuan Zhang, Xin Zhou, MingLiang Li, and Jun Wang. 2019. Recurrent Neural Network Based Binary Code Vulnerability Detection. In *Proceedings of the 2019 2nd International Conference on Algorithms, Computing and Artificial Intelligence (ACAI)*. ACM, Sanya, China, 160–165.

Received 2024-10-31; accepted 2025-03-31